

State Management with useState – React Hooks

Learning Objectives

Today You Will Learn

01

What is State?

02

useState Hook

03

Updating State

04

Object State

05

Array State

06

Controlled Components

07

Mini Projects

📌 **Learning Outcome:** By the end of today's session, you will be able to build interactive React applications where the UI updates automatically based on user actions.

What is State?

State

State is data that belongs to a component and can change over time.

Whenever state changes, React automatically updates the UI.

Real World Examples

- Counter Value
- Shopping Cart Quantity
- Login Status
- Form Input
- Theme (Dark/Light)
- Todo List

Example

Without State

```
let count = 0;
```

Changing count will **not** update the UI.

With State

```
const [count, setCount] = useState(0);
```

Changing state automatically updates the UI.

📌 **Key Point:** Props are Read Only. State is Changeable.

useState Hook

What is useState?

useState() is a React Hook used to create and manage state inside functional components.

Syntax

```
import { useState } from "react";  
const [count, setCount] = useState(0);
```

Breakdown

- `count` → Current Value
- `setCount` → Updates Value
- `0` → Initial Value

Example

```
import { useState } from "react";  
function Counter(){  
  const [count,setCount]=useState(0);  
  return(  
    <>  
    <h1>{count}</h1>  
    <button onClick={()=>setCount(count+1)}>  
      Increase  
    </button>  
    </>  
  )  
}
```

Updating State

Increment

```
setCount(count + 1);
```

Decrement

```
setCount(count - 1);
```

Reset

```
setCount(0);
```

Complete Example

```
const [count,setCount]=useState(0);  
  
<button onClick={()=>setCount(count+1)}> + </button>  
<button onClick={()=>setCount(count-1)}> - </button>  
<button onClick={()=>setCount(0)}> Reset </button>
```

❏ **Best Practice:** Use the previous state when the next value depends on it.

```
setCount(prevCount => prevCount + 1);
```

Object State

React can store objects inside state.

Object State Example

```
const [student, setStudent]=useState({  
  name:"Rahul",  
  course:"React",  
  age:22  
});
```

Update only one property

```
setStudent({ ...student, age:23 });
```

Why Spread Operator?

Without it,

```
setStudent({ age:23 });
```

Previous properties are removed.

Array State

Store multiple values inside an array.

Array State Example

```
const [students,setStudents]=useState([
  "Rahul",
  "Priya"
]);
```

Add a new student

```
setStudents([ ...students, "John" ]);
```

Display List

```
students.map((student,index)=>
  <li key={index}>{student}</li>
)
```

Real World

- Todo Lists
- Shopping Cart
- Product Lists
- Notifications

Controlled Components

What is a Controlled Component?

A controlled component is an input whose value is managed by React state.

Example

```
const [name, setName]=useState("");

<input
  type="text"
  value={name}
  onChange={(e)=>setName(e.target.value)}
/>
```

Display Input

```
<h1>Hello {name}</h1>
```

Benefits

Easy Validation

Real-Time Updates

Better Form Handling

Single Source of Truth

Mini Projects

Project 1. Counter

Features

- Increase
- Decrease
- Reset

Concepts

- useState
- Event Handling

Project 2. Todo List

Features

- Add Todo
- Display Todo
- Clear Input

Concepts

- Array State
- Controlled Component
- map()

Project 3. Shopping Cart Quantity

Features

- Product Quantity
- Increase
- Decrease
- Disable Negative Values

Concepts

- State
- Conditional Logic
- Event Handling

Live Coding — Counter Application

Counter App

```
import { useState } from "react";

function Counter(){
  const [count,setCount]=useState(0);
  return(
    <div>
      <h1>{count}</h1>
      <button
        className="btn btn-success me-2"
        onClick={()=>setCount(prev=>prev+1)}
      > + </button>
      <button
        className="btn btn-danger me-2"
        onClick={()=>setCount(prev=>prev-1)}
        disabled={count===0}
      > - </button>
      <button
        className="btn btn-secondary"
        onClick={()=>setCount(0)}
      > Reset </button>
    </div>
  )
}

export default Counter;
```

Concepts Used

useState

Event Handling

Conditional Button

Bootstrap Buttons

Homework Assignment

Homework 1. Student Attendance Tracker

Create an attendance tracker using React.

Each Student Should Display

- Student Name
- Roll Number
- Attendance Status

Features

-  Mark Present
-  Mark Absent
-  Toggle Attendance
-  Display Total Present
-  Display Total Absent

Submission Checklist

 useState Hook

 Object State

 Controlled Components

 Dynamic Styling

Challenge 1

Change card color dynamically.

- Present → Green
- Absent → Red

Challenge 2

Use Bootstrap Cards.

Challenge 3

Store all students in an array using useState().

```
const [students,setStudents]=useState([
  { id:1, name:"Rahul", present:true }
]);
```

Update attendance using map().

 State Updates

 Array State

 Bootstrap Cards

 Used map()



Expected Outcome: Students will understand how React manages state, update UI dynamically with useState, work with objects and arrays in state, build controlled forms, and create interactive applications such as counters, todo lists, shopping carts, and attendance trackers. These are essential React concepts used in almost every real-world application.